Video Based Soccer Ball Tracking

Sophie Xiaofan Liu†, Lijun Jiang*
†Engineering, Computer Science, Physics and Mathematics Department
Oral Roberts University
Tulsa Ok 74171
sliu@oru.edu

Jacob Garner†, Sharayah Vermette†
* School of Engineering
Republic Polytechnic, Singapore
9 Woodlands Avenue 9
Singapore 738964
jiang_lijun@rp.sg

***Abstract* - The purpose of this project is to design a system that is capable of locating and tracking a soccer ball during a normal soccer game. The necessary algorithms for this system accurately distinguish between stationary and moving objects in a stream of video frames and are able to consistently detect which of these moving objects is the soccer ball. The system is aware of the position of the soccer ball at all times. From the coordinates of the ball at a given time, the system is able to estimate where the ball will be located in the next few consecutive frames so as to ease the difficulty of tracking it. The resulting data can then be used to control a camera that will cover the action of the game, or to analyze the game afterwards.**

Keywords - soccer ball tracking, video analysis, foreground.

I. INTRODUCTION

The demand for computer assisted game study in sports is growing dramatically. Analysis of soccer video sequences has been an interesting application in computer vision and image analysis [1-4]. Much research has been done on ball and player detection and tracking. A template based approach was used to detect and track the soccer ball in [5]. In [6], a circle detection algorithm based on the circle Hough transform was implemented to detect the soccer ball. However, ball tracking has not been thoroughly studied because soccer ball tracking is known to be more difficult than that of players due to its small size in an image and abrupt changes in its motion. For example, D' Orazio et al.[6] and Matsumoto et al. [7] presented a modified Hough transform and template matching to detect balls in soccer videos, respectively. As irregular ball shapes are usually extracted at different velocities, these two methods are still insufficient.

In this paper, we will present a practical video analysis system that will greatly assist data collection and statistics analysis in soccer games. The following discussion provides details about all of the functions involved in this system.

II. BACKGROUND LEARNING

The background learning process sets up the initial background and the initial threshold for binarizing. The process primarily contains a loop that runs for each frame. The loop will run for the first few hundred frames to get a good base. The loop starts by reading the current frame from the video. It then averages all frames taken so far and makes this the background. The background value starts out as the first frame, and is updated each frame using a running average to build up the initial background. Continuing with the main loop, after the background is updated the rest of the loop is dedicated to creating an initial value for a variance image labeled *var(i,j,t)* for the pixel at *(i, j)* and time *t*. *Var(i,j,t)* will later be used to determine the threshold for determining if a pixel is part of the foreground or background. Let *bg(i,j,t)* and *cur(i,j,t)* be the background image and current image for the pixel at *(i, j)* and time *t* respectively. The variance image can be built up through the following equation.

$$\mathrm{var}(i,j,t) = \frac{[cur(i,j,t) - bg(i,j,t)] + (p-1) * \mathrm{var}(i,j,t-1)}{p} \quad (1)$$

where p is a weight that is a preset constant.

Figure 1. Two samples that are used for background learning.

Authorized licensed use limited to: East China Univ of Science and Tech. Downloaded on March 08,2026 at 00:42:01 UTC from IEEE Xplore. Restrictions apply.

Figure 2. Background image after the background learning.

III. Thresholding Using Standard Variation

The purpose of thresholding is to take a grayscale image and convert it to an image with only black and white elements. Establishing a threshold is a very important part of the decision process. There are several more sophisticated methods such as blob analysis [8] and morphological filtering. We developed a variance based threshold function because it is simple and time efficient in terms of implementation and updates with lighting changes. The function has three inputs and three outputs. The three inputs are labeled *bg(i,j,t-1)*, *cur(i,j,t)* and *var(i,j,t-1),* being the compiled grayscale background, the current frame and the pre-calculated background pixel variation, respectively. The outputs were labeled *fg(i,j,t), var(i,j,t)* and *bg(i,j,t),* being the determined binary foreground image, the updated background pixel variation and the updated background image, respectively.

A pixel in *cur(i,j,t)* is determined to be background if it satisfies the following criteria:

$$cur(i,j,t) > bg\ (i,j,t) - T(i,j,t) \tag{2}$$

$$cur(i,j,t) < bg\ (i,j,t) + T(i,j,t) \tag{3}$$

Next, a thresholding matrix is created by the following equation :

$$T(i,j,t) = var(i,j,t) * \alpha \tag{4}$$

T(i,j,t) is the threshold obtained by multiplying the pre-calculated value labeled *var(i,j,t)* by a preset constant α . This assigns a standard deviation to each pixel location, a value that provides a range for determining foreground regions from the stable background.

All pixels that do not fall into the range formed by equations (2) to (3) are considered to be foreground, with the value of 255 in *fg(i,j,t)*. If a pixel is determined not to be foreground, *fg(i,j,t)* is labeled as a black pixel, with the value of 0.

If the pixel is determined to be background, several things occur. First, the background needs to be updated. The basic idea of background updating is to establish a reliable background that is subtracted from the current image so that it looks similar to the background in the current frame of the image sequence. We update the background by taking a weighted average of the current background and the current frame of image sequence. The background image *bg(i,j,t)* is updated at the pixel location according to the following:

$$bg(i,j,t) = \frac{cur(i,j,t) + (q-1) * bg(i,j,t-1)}{q} \tag{5}$$

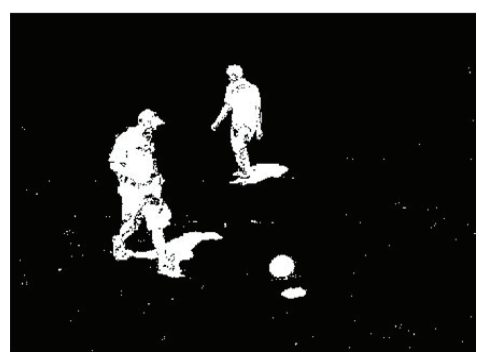
where q denotes a weight assigned to the current frame and background and it will affect the update speed.

Basically, equation (5) averages the current pixel's value in *cur(i,j,t)* with the existing value in the corresponding pixel in *bg(i,j,t)*. This serves as a constant update to keep the compiled background image current and stable.

Once *bg(i,j,t)* is updated at the pixel location, it needs to be determined if *var(i,j,t)* also needs to be updated. This is done by checking to see what *var(i,j,t)* would be recalculated as, and comparing it with a predetermined minimum value for *var(i,j,t)*. The equation used to update *var(i,j,t)* is equation (1).

The minimum value for *var(i,j,t)* is set at a value considered reasonable for the level of background stability available, currently at 8 for our purposes. If the newly calculated *var(i,j,t)*, as determined by equation (1), is below the minimum value set for *it*, *var(i,j,t)* will not keep the updated value. It will remain as it was previously. Doing these two steps allows *bg(i,j,t)* to only be updated if the pixel is not considered to be a foreground region, thus stabilizing it as accurately as possible.

Once each pixel in *cur(i,j,t)* has been examined in this way, the function outputs its results. The resulting *fg(i,j,t)* is an image with only black and white regions. The white regions are those that experienced significant deviation from the compiled background, *bg(i,j,t).* The updated *var(i,j,t)* will be used in the comparison of the next consecutive frame. The resulting *bg(i,j,t)* is the updated version that will be passed to the next iteration of the function.



Authorized licensed use limited to: East China Univ of Science and Tech. Downloaded on March 08,2026 at 00:42:01 UTC from IEEE Xplore. Restrictions apply.

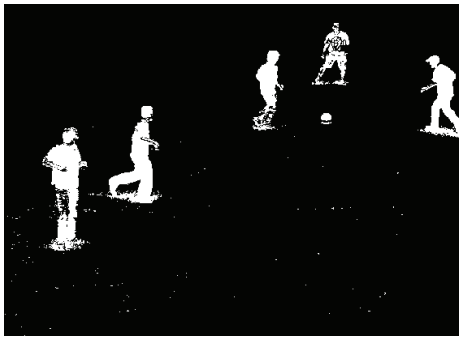


Figure 3. The Two Sample Frames After Thresholding.

IV. Remove Unwanted Noise

The purpose of this process is to remove the unwanted white specks that are considered to be noise. At the same time, all regions that happened to be separated into several regions due to the thresholding will be reconnected as a whole region, if appropriate.

We developed a morphological function to complete this task. The function has one input and one output. The input is foreground image *fg(i,j,t)*, and is the binary image resulting from the thresholding. The output is labeled *m1(i,j,t)*, and is the resulting de-noised image.

This function uses specific parameters for dividing the field seen in *fg(i,j,t)*. The boundaries apply to an image *W x H*, width by height. Due to the images being taken at an angle relative to the field, the proportions of the objects in the image changed based on how high or low they were in the image. For example, an object at the bottom of the image would be larger than the same object at the top of the image. Also, the scale of the objects changed depending on the vertical position in the image. For example, an object in the lower half of the image could move to a slightly higher position and experience minimal change in proportion. However, an object in the upper half of the image would experience significant change in proportion if it had the identical amount of position change. The higher the object is in the image, the more change in proportion it will experience. To take this into account, we have initially divided the image into four horizontal segments. For an image that is *H* pixels high, the segments have the following ranges:

$$0 < I \le \frac{H}{6}$$
$$\frac{H}{6} < I \le \frac{H}{3}$$
$$\frac{H}{3} < I \le \frac{2H}{3}$$
$$\frac{2H}{3} < I \le \frac{H}{6}$$

with *I* being the row index, or the vertical position of the pixel. Based on these ranges, the function determines what parameters to apply to each pixel. These parameters are different for each of the four passes that it executes.

To begin, the function examines each individual pixel. The purpose of the first pass is to search for strings of horizontal and vertical black pixels that are shorter in length than a given predetermined parameter, or value. If the current pixel in the image, *fg(i,j,t)* is equal to 0, or is black, a counter *n* is incremented by one. If it is not equal to 0, then the function checks to see if *fg(i,j-1,t)* is equal to 0. Basically, this means that the function keeps a count of consecutive black pixels until it encounters a white pixel, or a pixel with 255. When a string of black pixels is ended by a white pixel, the function checks to see if the length of the black pixel string is less than the given parameter for the vertical segment. If it satisfies the parameter, the string of black pixels is considered to be foreground, inaccurately labeled beforehand in *binary*, and the entire string of pixels is turned to white. This process is executed for each row, and then again for each column. The parameter for this first pass through *fg(i,j,t)* is generally small, with the main focus being to fill in the small black gaps within the large white regions. The result of this first pass is *fg(i,j,t)* with more complete white regions.

The second pass over *fg(i,j,t)* with the function is to find pieces of unwanted noise, or individual white pixels not connected to any significant region, throughout the image and change them to black pixels. This is achieved in the same way as the first pass. The difference is that now the function looks for a short string of white pixels that satisfies the given parameter. For this pass, the parameters are generally larger for all vertical sections of the frame than they were for the first pass. This is because the initial black-to-white pass may have possibly enlarged the small sections of noise in the image. So a larger parameter to remove these sections of noise is needed. The strings of white pixels are examined both horizontally and vertically. The result of this pass is *fg(i,j,t)* with all unwanted noise removed from the image, while still keeping most large regions intact.

As a result of the second pass, some of the desired foreground regions were disconnected at their narrower sections. Thus, some regions that used to be whole might now be split into several smaller regions. To account for this happening, the function has a third and fourth pass over *fg(i,j,t)* similar to the first pass, the third and fourth pass use the same method of counting strings of black pixels. However, this time, each pass only examines the pixels in one direction, either vertical or horizontal. The third pass handles the vertical strings of pixels, and uses a relatively large parameter to determine which strings of black pixels should be turned to white. This increases the chance of regions being reconnected vertically that had been separated previously. The fourth pass handles the horizontal strings of pixels, and uses a much smaller parameter to determine which strings of black pixels should be turned to white. This allows nearby regions that may have been slightly separated from a larger region to be reconnected horizontally.

The resulting *fg(i,j,t)* is outputted as *m1(i,j,t)* and is an image with clear, distinct regions that can be recognized for future use.

Authorized licensed use limited to: East China Univ of Science and Tech. Downloaded on March 08,2026 at 00:42:01 UTC from IEEE Xplore. Restrictions apply.

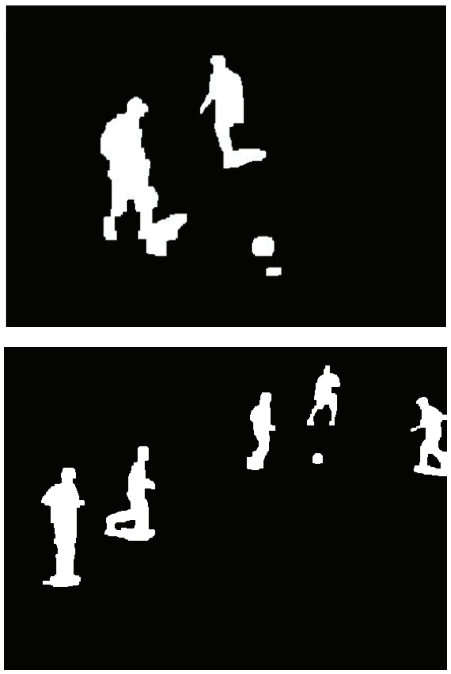


Figure 4. Two Samples After Morphological Function.

V. BALL DETECTION

The ball detection function detects the ball in a single frame. It works when the ball is completely separate from any players. First a data matrix *A* is created to reflect the connections between various region numbers in one frame. The function begins by creating a new matrix, *U*, which will have a row for each region in the frame. The 4-column rows will initially only contain the region number of that row, and three empty spaces. The function then fills these spaces with the size of the region in pixels, and the average vertical and horizontal positions of the pixels in the region. The function now only uses *U* to examine each region. Each region's size is now checked against a typical ball size range for that location in the frame. If a region is found to be approximately the size of a ball for that position, that region will be marked as the ball. The function can also output the location of the center of the ball if needed.

There may be times when multiple regions could be marked as the ball. For example, if a person is partially outside of the frame, a portion of his body that is in the frame might be the same size as the ball. We allow for this because sometimes the size of the ball can fluctuate, such as if it is connected to its shadow. We also added a second step in the function that checks the ball candidate regions from the first step and decides which region is most likely the ball by looking at the shape. A ball is spherical, so even with noise removal taking out parts of the ball, it generally stays as wide as it is tall. We checked the widest and tallest parts of each candidate region, and decide which region was most square. This region has the greatest chance of being the ball.

VI. CONCLUSION

The algorithms have been tested on many different videos, and they work well. Detection rates are adequate for a typical soccer game. There will be times when the ball is partially or completely obscured by a player, and there will be times when the ball simply is not even in the frame being analyzed. To account for this, our future work is to create a simplistic function, *Predictor*. The function *Predictor* will provide extra ball location data when there is no *Detector* data available.

REFERENCES

[1] X.F. Tong, H.Q. Lu, Q.S. Liu, "An effective and fast soccer ball detection and tracking method," Proc. ICPR, 2004, pp. 795–798.

[2] X. Yu, C. Xu, Q. Tian, and H. W. Leong, "A Ball Tracking Framework for Broadcast Soccer Video," ICME, Baltimore, US, July 6-9, 2003.

[3] D. Liang, Y. Liu, Q. Huang, W. Gao, "A scheme for ball detection and tracking in broadcast soccer video, " Proc. Pacific Conference Multimedia, LNCS 3767, 2005, pp. 864–875.

[4] H-S, Yoon, Y-L J, Y-K Yang, "A soccer image sequence mosaicking and analysis method using line and advertisement board detection, *ETRI Journal*, Vol. 24, No. 6, 2002, pp.443-454.

[5] D. Yow, B. Yeo, M. Yeung and B. Liu, "Analysis and presentation of soccer highlights from digital video," *Second Asian Conf. on Compute Vision*, Dec. 1995, pp. 499-503.

[6] T. D'Orazio, C. Guaragnella, M. Leo, A. Distante, "A new algorithm for ball recognition using circle Hough transform and neural classifier," Pattern Recognition 37 (3), 2004, pp. 393–408.

[7] K. Matsumoto, S. Sudo, H. Saito, S. Ozawa, "Optimized camera viewpoint determination system for soccer game broadcasting," Proc. IAPR Workshop Machine Vision Appl., Tokyo, 2000, pp. 115 – 118.

[8] R. Leitner, H. Mairer and A. Kercek, "Real-time classification of polymers with NIR spectral imaging and blob analysis," Real-Time Imaging, Vol. 9, Issue 4, August 2003, pp. 245-25, doi:10.1016/j.rti.2003.09.016.

Authorized licensed use limited to: East China Univ of Science and Tech. Downloaded on March 08,2026 at 00:42:01 UTC from IEEE Xplore. Restrictions apply.